

24皇后問題共有227,514,172,009,468個解

謝育平^{a*}, 項潔^{a†}, 黃光璿^b, 許德標^c

^a國立台灣大學 資訊工程學系

^b國立暨南大學 資訊工程學系

^cDepartment of Computer and Information Science, Fordham University.

arping@turing.csie.ntu.edu.tw

hsiang@csie.ntu.edu.tw

shieng@ncnu.edu.tw

hsu@trill.cis.fordham.edu

Abstract

n -Queen 問題已經被討論至少 150 年, 產出至少 88 篇相關的論文, 衍生出很多的變形問題並激發出各式不同的演算法。其中最受人矚目的還是 n -Queen 原始問題的存在性問題及計數性問題。本文嘗試有別於以往的演算法, 使用 Divide and Conquer 的概念來處理計數性問題, 並計算出 24-Queen 的解個數。

1 Introduction

n -Queen 問題[1, 3, 13] 已經被討論至少 150 年, 產出至少 88 篇相關的論文[11], 衍生出很多的變形問題[5, 7], [20](Queen) (在[20] 的網站中, 選取 “Word” 並查詢 “Queen”, 會出現 52 個有關 n -Queen 問題的數字數列), 其中最受人矚目的還是 n -Queen 問題本身的解存在性問題[1, 2] 及解計數性問題[17, 5, 7], [20](A000170) (在[20] 的網站中選取 “Sequence number”, 查詢 “A000170”, 即可得到 n 從 1 到 23 的 n -Queen 問題解的個數)。另外兩個跟 n -Queen 問題有極大關係的是 Toroidal(Modular) n -Queen Problem [20](A007705) 及 Toroidal-Semi n -Queen Problem[20](A006717)。

我們最早開始這方面的興趣是起於研究 complete mapping [12, 6, 15, 8]。而 Complete Mapping 跟 n -Queen Problem 正有著密不可分的關係。一個 cyclic group $(Z_n, +)$ 的 Complete Mapping 就是一個 Toroidal-Semi n -Queen Problem 的解, 但是 Complete Mapping 多了一個條件就是第一個 Queen 必須放在第一個位置($f(0) = 0$), 所以 Toroidal-Semi n -Queen Problem 的解個數剛好是 Complete Mapping 個數的 n 倍。而一個 Z_n group 的 Strong Com-

plete Mapping [9] 就是一個 Toroidal n -Queen Problem 的解, 但是多了一個條件就是第一個 Queen 必須放在第一個位置, 所以 Toroidal n -Queen Problem 的解個數也剛好是 Strong Complete Mapping 個數的 n 倍。我們將計算 Complete Mapping 個數的技巧[19, 7] 應用在計算 24-Queen 問題上, 並加上一些 n -Queen 問題特有的技術, 嘗試使用 Divide and Conquer 的概念來處理, 並解決 24-Queen 的計數性問題。

2 n -Queen問題

n -queen 計數問題: 給定一個 n 乘 n 的棋盤, 有多少種方法可以將 n 個皇后放進棋盤中, 使得每一行、每一列、每一個 (左上右下) 斜線和 (右上左下) 斜線中, 至多一個皇后。我們將一個解答 (擺設的方法) 用一個對射函數 (1-1, onto) $f: Z_n \rightarrow Z_n$ 紀錄之。下圖表示了 8-Queen 問題的一個解。

f	0	1	2	3	4	5	6	7	f	$f - id$	$f + id$
0		Q							1	1	1
1	\							Q	7	8	6
2		\				Q		/	5	7	3
3	Q		\				/		0	3	-3
4			Q	\		/			2	6	-2
5	—	—	—	—	Q	—	—	—	4	9	-1
6				/		\	Q		6	12	0
7			/	Q			\		3	10	-4

Figure 1: 8-Queen 問題的一個解。

所以我們用更數學的方式來描述 n -Queen 問題: 一個 n -Queen 問題的解是一個從 Z_n 對到 Z_n 的函數

*感謝國科會計畫 NSC 92-2811-E-002-027 的支持

†感謝國科會計畫 NSC 92-2213-E-002-005 的支持

$f(x)$ 使得

$$\begin{aligned}\forall i \neq j \in Z_n \quad & f(i) \neq f(j), \\ \forall i \neq j \in Z_n \quad & f(i) - i \neq f(j) - j, \\ \forall i \neq j \in Z_n \quad & f(i) + i \neq f(j) + j\end{aligned}$$

(此處的加減是指整數下的加法)。而 n -Queen 的計數問題則是探討有多少個解。我們用 $Q(n)$ 來表示解的個數。我們很容易地可以發現將原來的解隨著棋盤旋轉 90° 後也會是一個 n -Queen 問題的解。我們稱這樣的一個動作 (旋轉 90°) 為一個對稱運算子。定義如下: π 是作用在解答上的一個對稱運算子 (symmetry operator): 如果 f 是一個解則 $\pi(f)$ 也是一個解, 反之亦然。對於 n -Queen 問題, 有八個很自然的對稱運算子由兩個基本運算子所延展[14, 4]。

1. 一個是逆時鐘旋轉 90° :
 $R_{90^\circ}(f)(x) = (n - 1 - f)^{-1}(x)$,
2. 另一個是依主對角線 (左上右下) 翻轉:
 $R(f)(x) = f^{-1}(x)$ 。

根據 Integer Sequence[20] 上記載, 及本文的結果, 我們將 n -Queen 解的個數列表如下。

n	$Q(n)$
1	1
2	0
3	0
4	2
5	10
6	4
7	40
8	92
9	352
10	724
11	2,680
12	14,200
13	73,712
14	365,596
15	2,279,184
16	14,772,512
17	95,815,104
18	666,090,624
19	4,968,057,848
20	39,029,188,884
21	314,666,222,712
22	2,691,008,701,644
23	24,233,937,684,440
24	227,514,172,009,468

在本文之前, n -Queen 計數問題, 最好的結果為 Sylvain Pion 及 Joel-Yann Fourre [16] 所計算的 $Q(23)$ 。

3 計算 $Q(24)$

對於計算 $Q(24)$ 來說, 我們主要採取 Partition Strategies[18] 為主, symmetry cutting strategies[10] 為輔。首先我們觀察以下的定義及定理, 我們依照原本 n -Queen Problem 解的定義找出其若且唯若的九個條件, 並將這九個條件應用在程式寫作上。

定義 1. 若 $f: Z_n \rightarrow Z_n$ 滿足以下三個條件, 則我們稱 f 為 n -Queen 問題的一個解。

1. $\forall i \neq j \in Z_n \quad f(i) \neq f(j)$,
2. $\forall i \neq j \in Z_n \quad f(i) - i \neq f(j) - j$ 和
3. $\forall i \neq j \in Z_n \quad f(i) + i \neq f(j) + j$ 。

此處的加減法是整數的加減法 $(Z, +)$, 並非是同餘群下的加減法 $(Z_n, +)$ 。

定理 1. 假設 U 是 Z_n 的一個子集, 取 $L = Z_n \setminus U$ 為 Z_n 對 U 的差集。則 $f: Z_n \rightarrow Z_n$ 是 n -Queen 問題的一個解若且唯若 f 滿足以下九個條件。

1. $\forall i \neq j \in U \quad f(i) \neq f(j)$
2. $\forall i \neq j \in U \quad f(i) - i \neq f(j) - j$
3. $\forall i \neq j \in U \quad f(i) + i \neq f(j) + j$
4. $\forall i \neq j \in L \quad f(i) \neq f(j)$
5. $\forall i \neq j \in L \quad f(i) - i \neq f(j) - j$
6. $\forall i \neq j \in L \quad f(i) + i \neq f(j) + j$
7. $f(U) \cap f(L) = \emptyset$
8. $(f - id)(U) \cap (f - id)(L) = \emptyset$
9. $(f + id)(U) \cap (f + id)(L) = \emptyset$

其中 $f(S) = \{f(x) \mid x \in S\}$, $(f - id)(S) = \{f(x) - x \mid x \in S\}$, $(f + id)(S) = \{f(x) + x \mid x \in S\}$ 。

3.1 基本架構

所以在計算 $Q(24)$ 時, 我們先將 Z_{24} 拆成兩部分, $U = \{0, 1, 2, \dots, 11\}$, $L = \{12, 13, 14, \dots, 23\}$, 對於一個 24-Queen 解 $f(x)$ 來說, $f(U) = \{f(i) \mid i \in U\}$ 是固定的, 而且 $|f(U)| = 12$, 所以我們依照 $f(U)$ 的不同, 將所有的解分成 C_{12}^{24} 個 partition slots, 作為分割計算的方法。在 $f(U)$ 決定後, 我們使用條件 4 及條件 7, $f(L)$ 也會被唯一決定。所以我們的演算法架構如下:

令 number=0 來記載 Q(24) 的值
 令 $U = \{0, 1, 2, \dots, 11\}$, 而 $L = Z_n \setminus U$
 For each partition slot $A \subset Z_n, |A| = 12$
 1. 令 $f(U) = A, f(L) = Z_n \setminus A$.
 此時條件 7 自然滿足。
 2. 產生所有的部分解 $f : U \rightarrow f(U)$
 使得條件 1, 2, 3 都成立並紀錄所有的 U-record:
 $((f - id)(U), (f + id)(U))$.
 3. 產生所有的部分解 $f : L \rightarrow f(L)$
 使得條件 4, 5, 6 都成立並紀錄所有的 L-record:
 $((f - id)(L), (f + id)(L))$.
 4. 將所有的 U-record 對所有的 L-record
 進行比對, 檢查條件 8,9 是否成立,
 若成立則將 number 加 1。
 End

3.2 使用對稱運算子加速計算

此時我們引用 symmetry cutting strategies, 使用對稱運算子來加速計算。因為我們使用 partition strategies 為主的關係導致並非八個對稱運算子都可以拿來加速計算, 只有上下翻轉 ($R_{90^\circ} \circ R$) 和左右翻轉 ($R_{270^\circ} \circ R$) 所延展的四個對稱運算子可供使用。也就是說在計算上, 將大致有四倍的加速。這裡我們強調一下, 如何運用 symmetry cutting strategies 會因 n 的奇偶性而有所不同, 所以以下的整個演算法架構是針對偶數 n 而設計的。

定義 2. 假設 A, B 為 Z_n 的兩個子集, 而 h 為 Z_n 的一個元素。

1. $h - A = \{h - y \mid y \in A\}$.
2. $A < B$ if there exists an $x \in A \setminus B$ such that $x < y$ for all $y \in B \setminus A$.

引理 1. 令集合 ordering $<$ 的定義如上, 則對於所有 Z_n 的子集 A, B , 則 $A = B, A < B, B < A$ 恰一成立。

定理 2. 假設 $n = 2k, f : Z_n \rightarrow Z_n$ 為 n -Queen 問題的一個解。令 $U = \{0, 1, 2, \dots, k-1\}, L = Z_n \setminus U = \{k, k+1, \dots, 2k-1\}$ 。對於 n -Queen 問題的上下翻轉運算子 $\pi_1 = R_{90^\circ} \circ R$, 和左右翻轉運算子 $\pi_2 = R_{90^\circ} \circ R$, 以下性質成立。令 $A = f(U)$ 則

1. $\pi_1(f)(U) = Z_n \setminus A$.
2. $\pi_2(f)(U) = (n-1) - A$.
3. $\pi_2 \circ \pi_1(f)(U) = \pi_1 \circ \pi_2(f)(U) = (n-1) - (Z_n \setminus A)$.
4. $\min\{A, Z_n \setminus A, (n-1) - A, (n-1) - (Z_n \setminus A)\}$ 存在且唯一。

令 number=0 來記載 Q(24) 的值
 令 $U = \{0, 1, 2, \dots, 11\}$, 而 $L = Z_n \setminus U$
 For each partition slot $A \subset Z_n, |A| = 12$
 a. 令 factor =
 $|\{A, (n-1) - A, Z_n \setminus A, (n-1) - Z_n \setminus A\}|$
 b. 如果 $A \neq$
 $\min\{A, (n-1) - A, Z_n \setminus A, (n-1) - Z_n \setminus A\}$
 則繼續下一個 partition slot。
 1. 令 $f(U) = A, f(L) = Z_n \setminus A$.
 此時條件 7 自然滿足。
 2. 產生所有的部分解 $f : U \rightarrow f(U)$
 使得條件 1, 2, 3 都成立並紀錄所有的 U-record:
 $((f - id)(U), (f + id)(U))$.
 3. 產生所有的部分解 $f : L \rightarrow f(L)$
 使得條件 4, 5, 6 都成立並紀錄所有的 L-record:
 $((f - id)(L), (f + id)(L))$.
 4. 將所有的 U-record 對所有的 L-record
 進行比對, 檢查條件 8,9 是否成立,
 若成立則將 number 加 factor。
 End

3.3 使用 bit tree 加速計算

在上述的所有步驟中以步驟 4 為時間的瓶頸, 所以我們將步驟 4 做些改善。在步驟 2 或 3 中, 因為 $f - id$ 的值域為 $[-23, 23]$ 而 $f + id$ 的值域為 $[0, 46]$, 所以紀錄 U-record 或是 L-record 須要 94 bits。

令 $x \in U, y \in L$, 則 $0 \leq x \leq 11, 12 \leq y \leq 23$, 所以 $-11 \leq f(x) - x \leq 23, -23 \leq f(y) - y \leq 11$, 所以要計算 $(f - id)(U), (f - id)(L)$ 是否互斥只需要檢查 $(f - id)(U) \cap [-11, 11]$ 和 $(f - id)(L) \cap [-11, 11]$ 是否互斥即可。故我們使用 23 bits 來記載 $(f - id)$ 的部分。同理 $0 \leq f(x) + x \leq 34, 12 \leq f(y) + y \leq 46$, 所以要計算 $(f + id)(U), (f + id)(L)$ 是否互斥只需要檢查 $(f + id)(U) \cap [12, 34]$ 和 $(f + id)(L) \cap [12, 34]$ 是否互斥即可。故我們也須使用 23 bits 來記載 $(f + id)$ 的部分。所以我們不需要記載完整的 record 資訊, 我們只需使用 46 bits 來紀錄 U-record 和 L-record。

接下來使用 46 bits 檢查條件 8, 9 是整個演算法的瓶頸, 是最耗時的一段程式區碼。令 US 分別為步驟 2 所產生的所有 U-record 所形成的 multi-set。令 LS 為步驟 2 所產生的所有 L-record 所成的 multi-set。此時若我們將 US 中的每個 record 拿來跟 LS 中的每個 record 做比對來檢查條件 8, 9, 此時所需要的比對為 $|US| \times |LS|$ 次, 所以如果可以在這裡取得突破則可以大大加速計算。

接下來我們將 US 中的 record 依照 bit0 分兩區 US_0, US_1 , 每一區再依照 bit1 再分兩區成 $US_{00}, US_{01}, US_{10}, US_{11}$ 。如此建構成一棵樹來記載 US, 每個 leaf node 都表示一個 record, 而每個 internal node 都表示一群 records。同理將 LS 用一棵樹來表達。此時紀錄 record 的 bit 數及其順序對建樹所使用的記憶體及條件 8, 9 的檢查時間有很大的影響, 這也就是為什麼我們要將 bit 數從 94 降為 46。

然後當我們在比對 record 來檢查條件 8, 9 時, 就變成了一個遞迴的過程。在這個過程中, 我們得到了一些好處。

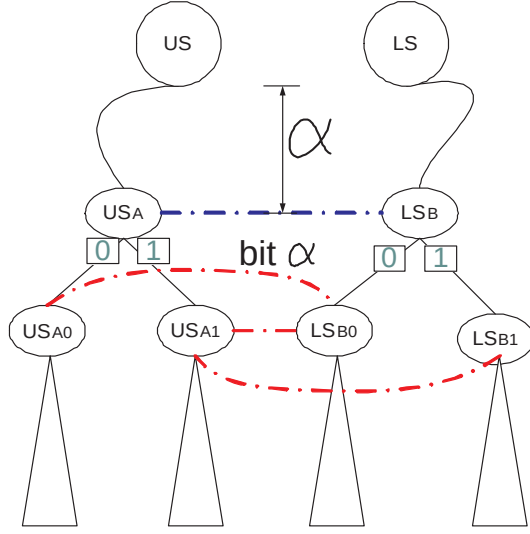


Figure 2: Record比對狀況。

令 $A, B \in \{0, 1\}^*$ 為從根節點到某節點的路徑且 A, B 長度皆為 α , 則 US_A, LS_B 分別代表 US 及 LS 樹的某個節點, 也就是分別代表一群 U-record 及 L-record。此時因為條件 8, 9 要滿足, 蘊含 U-record 和 L-record 在 $bit\alpha$ 不可皆為 1。所以當我們要比對這兩群 (US_A, US_B) 滿不滿足條件 8, 9 時, 只需比對 $(US_{A0}, LS_{B1}), (US_{A1}, LS_{B0}), (US_{A0}, LS_{B0})$ 而不需比對 (US_{A1}, LS_{B1}) 所以省下了很多寶貴的時間。

我們假設每個 US 或 LS 的 record 記載的 46 bits 資訊中, 每個 bit 被設 1 的機率大致相同, 在此假設下, 我們約略估算每個 bit 被設 1 的機率為 $(12 + 12)/(47 + 47)$ 約為 0.25, 這表示當我們在比對 (US_A, LS_B) 時, US_{A1} 和 US_{A0} 的元素個數比約為 $0.25 : 0.75 = 1 : 3$, 同樣地, LS_{B1} 和 LS_{B0} 的元素個數比約為 $1 : 3$ 。所以當我們在比對 (US_A, LS_B) 時我們節省了比對 (US_{A1}, LS_{B1}) 的時間, 所以我們可以省下約 $\frac{1}{4} \times \frac{1}{4} = \frac{1}{16}$ 的時間。由於這個演算法是遞迴地在處理, 所以省下的時間是 $\frac{1}{16} \times (1 + \frac{15}{16} + (\frac{15}{16})^2 + \dots + (\frac{15}{16})^{45}) = 0.95$, 則所花的時間就是 $1 - 0.95 = 0.05$, 幾乎加速 20 倍。我們這裡使用了每個 bit 被 set 1 的機率大致相同的假設, 事實上, 我們可以將設 1 頻率越高的 bit (棋盤中越中間的部位) 排越前面, 然後再進行建樹的工作, 此項安排將會再加速本演算法。

在實驗上, 下表列出平均每個 partition slot 在步驟 4 中所花的時間, 其中區分為使用或未使用 bit-tree 技術的時間 (單位為 megacycles), 我們並列出兩個時間的比與 $(\frac{15}{16})^{2n-2}$ 數值的比較。值得觀察的是, 使用 bit-tree 技術的時間與未使用該技術的時間比, 果然從 $n = 6$ 的 2.00 降到 $n = 20$ 的 0.15, 雖然 $(\frac{15}{16})^{(2n-2)}$ 是我們理論上期待的數值, 但其差距亦隨著 n 的增大而縮小, 所以

猜測 $n = 24$ 的所花時間比會小於 $(\frac{15}{16})^{(2n-2)}$ 。

n	使用	未使用	使用/未使用	$(\frac{15}{16})^{2n-2}$
6	1.00E-03	5.00E-04	2.00	0.52
8	1.00E-03	4.29E-04	2.33	0.41
10	2.46E-03	1.27E-03	1.94	0.31
12	1.47E-02	7.36E-03	2.00	0.24
14	1.23E-01	9.12E-02	1.35	0.19
16	1.31E+00	1.93E+00	0.68	0.14
18	1.84E+01	5.50E+01	0.33	0.11
20	2.93E+02	2.01E+03	0.15	0.09
22	5.31E+03	未算出	未算出	0.07
24	1.11E+05	未算出	未算出	0.05

4 實驗結果與討論

我們使用平行計算將 $Q(24)$ 問題依 $C_{12}^{24} = 2,704,156$ partition slots 分成 1000 jobs, 然後以 job 為單位分送各台 PC 計算。在各台 PC 中, 我們將計算程式的執行優先序設至最低, 以免影響其他同仁的工作, 雖然如此, 每台機器大抵都有 90% 以上的時間是在執行 24-Queen 的計算。我們的演算法在每台機器使用了 800 Mega Bytes 記憶體, 成功地在 322,730,315,531.47 mega cycles (約 1867 個 P4-2.0GHz PC 工作天) 算出 $Q(24) = 227,514,172,009,468$ 。

Figure 3 是 20-Queen, 22-Queen, 24-Queen 問題中, 我們演算法的執行各 job 的時間圖表。在三張圖表中, 大抵 job id 比較低的, 執行時間比較久, 這是因為使用 symmetry cutting 策略的關係, 使得後面有很多 partition slot 不需要計算的原因。除了 24-Queen 問題有些瑕疵之外, 三張圖除了時間規模之外, 圖形走勢非常地相似。24-Queen 問題中的幾個超出預期執行時間的 job 可能肇因於某 PC 同時間有其他同仁在大量使用 CPU 時間或是該電腦受病毒影響拖累 CPU 使用效能。

Pion 利用 Backtracing 的方法, 從 $i = 0$ 到 $i = n - 1$ 逐一設定 $f(i)$ 的值, 然後用 “bitwise and” 和 “bitwise shift” 的方法來檢驗 $\forall i \neq j \in Z_n f(i) \neq f(j)$, $\forall i \neq j \in Z_n f(i) - i \neq f(j) - j$, $\forall i \neq j \in Z_n f(i) + i \neq f(j) + j$, 是一個標準且直覺的做法, 但是因為 Pion 的巧思設計 bitwise operator 的運作使得計算速度大幅提升。Pion 並且利用平行計算算出 $Q(23)$ 。Ken'ichiro Takahashi[21] (感謝 Patrick René Guillemin 告知) 利用 Pion 的方法加上使用 8 個 Symmetry operator 來做 symmetry cutting 使得速度再次提升。

我們重新製作 Pion 的方法進行實驗合併 Takahashi 在網頁上的數據整理如下, 所需時間是以 2.0GHz PC 為依據, 其中時間數據格式為 (day hour:minute:second)。數據中值得觀察的是 Pion's Algorithm 所需時間與 Takahashi's Algorithm 所需時間大致成正比, 但是 Partition Strategy 所需時間與 Takahashi's Algorithm 所需時間的比則從 $n = 16$ 的 5 倍下降到 $n = 18$ 的 3 倍多再降到 $n = 20$ 的 2 倍多, 這意味著 Partition Strategies 在 $n = 22$ 或

$n = 24$ 時將取得稍微的領先。

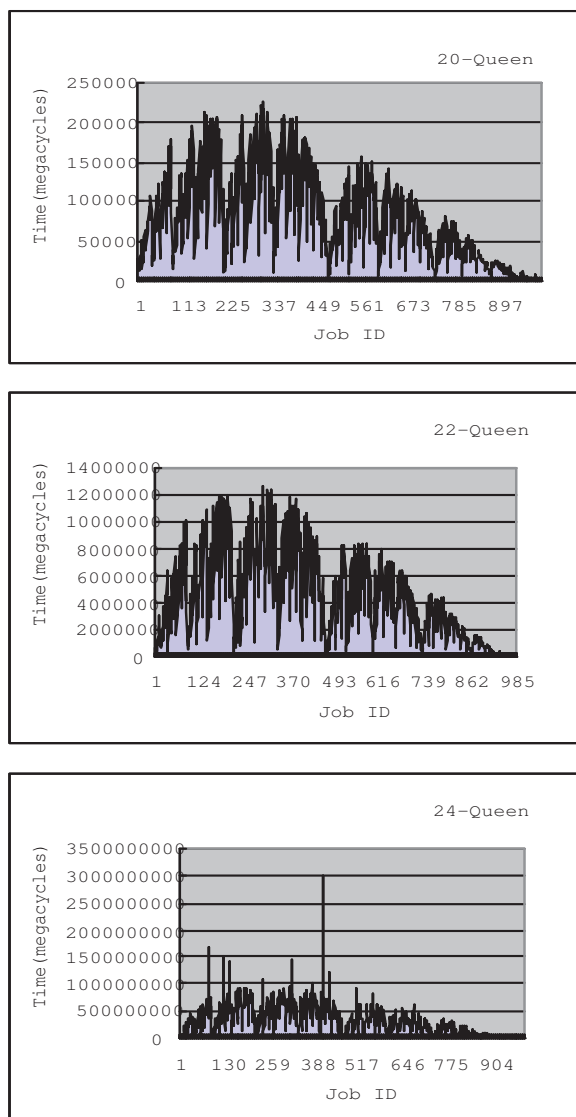


Figure 3: 計算 20, 22, 24-Queen 問題中各 job 的時間。

n	Pion 的演算法 所需時間	Takahashi 的演算法 所需時間	本演算法 所需時間	建樹 記憶體 (KB)
14	0 00:00:01	0 00:00:00	0 00:00:01	92
15	0 00:00:08	0 00:00:01		
16	0 00:00:57	0 00:00:06	0 00:00:30	523
17	0 00:06:21	0 00:00:41		
18	0 00:48:12	0 00:04:56	0 00:16:05	2603
19	0 06:03:39	0 00:37:50		
20	2 01:38:23	0 05:03:35	0 11:09:44	15656
21		1 18:40:00		
22			25.75天	104207
23				
24			1867天	795420

References

- [1] B. ABRAMSON AND M. YUNG, *Divide and conquer under global constraints: A solution to the n -queens problem*, Journal of Parallel and Distributed Computing, 6 (1989), pp. 649–662.
- [2] W. AHRENS, *Mathematische unterhaltungen und spiele*, B. G. Teubner, Leipzig, (1910).
- [3] W. W. R. BALL, *Mathematical recreations and essays*, MacMillan and Co., (1926), p. 113.
- [4] P. CULL AND R. PANDY, *Isomorphism and the n -queens problem*, SIGCSE Bulletin, (1994), pp. 26:29–36.
- [5] C. ERBAS, S. SARKESHIK, AND M. M. TANIK, *Different perspectives of the n -queens problem*, in Proceedings of the ACM 1992 Computer Science Conference, 1992.
- [6] A. B. EVANS, *Orthomorphism graphs of groups*, vol. 1535 of Lecture Notes in Mathematics, Springer-Verlag, 1991.
- [7] J. HSIANG, Y.-P. SHIEH, AND Y.-C. CHEN, *The cyclic complete mappings counting problems*, in Problems and Problem Sets for ATP, volume 02-10 of DIKU technical reports, G. Sutcliffe, J. Pelletier, and C. Sutner, eds., July 31 - Aug 1 2002.
- [8] D. F. HSU, *Orthomorphisms and near orthomorphisms*, in Graph Theory, Combinatorics, and Applications (Y. Alavi eds.), John Wiley and Sons, Inc., 1991, pp. 667–679.
- [9] D. F. HSU AND A. D. KEEDWELL, *Generalized complete mappings, neofields, sequenceable groups and block designs II*, Pacific J. of Math., 117 (1985), pp. 291–312.
- [10] G.-S. HUANG, *Search Reduction Techniques and Applications to Problems in Combinatorics*, PhD thesis, National Taiwan University, 1999.
- [11] W. KOSTERS, *N-Queens* <http://www.liacs.nl/home/kosters/nqueens.html>.
- [12] H. B. MANN, *The construction of orthogonal Latin squares*, Ann. Math. Statistics, 13 (1942), pp. 418–423.

- [13] F. NAUCK, *Schach*, Illustrierter Zeitung, 361 (1850), p. 352.
- [14] A. T. OLSON, *The eight queens problem*, Journal of Computers in Mathematics and Science Teaching, (1993), p. 12:93.
- [15] L. J. PAIGE, *A note on finite abelian groups*, Bull. Amer. Math. Soc., 53 (1947), pp. 590–593.
- [16] S. PION AND J.-Y. FOURRE, *N-Queen Program* <http://www-sop.inria.fr/prisme/personnel/pion/progs/reines.tgz>.
- [17] I. RIVIN, I. VARDI, AND P. ZIMMERMANN, *The n-queens problem*, vol. 101 of Amer. Math. Monthly, 1994, pp. 629–639.
- [18] Y.-P. SHIEH, *Partition Strategies for #P-Complete Problems with Applications to Enumerative Combinatorics*, PhD thesis, National Taiwan University, 2001.
- [19] Y.-P. SHIEH, J. HSIANG, AND D. F. HSU, *On the enumeration of abelian K-complete mappings*, vol. 144 of Congressus Numerantium, 2000, pp. 67–88.
- [20] N. J. A. SLOANE, *Integer Sequence* <http://www.research.att.com/~njas/sequences/>.
- [21] K. TAKAHASHI, *N-Queen Problem* <http://www.ic-net.or.jp/home/takaken/e/queen/index.html>, 2003.